Secure and Role-Aware Explainable Occupant Tracking in Multi-Building Indoor Smart Spaces

Project report submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

Submitted by

Ajith Ashok AM.U4.U4CSE23204

Arjun Rajesh AM.U4.U4CSE23208

D Aditya Kiran AM.U4.U4CSE23215

Yadhu Vipin Madhilakath Madathil AM.U4.U4CSE23270

Under the guidance of

Dr. Lakshmi Mohan

Assistant Professor



AMRITA SCHOOL OF COMPUTING

AMRITA VISHWA VIDYAPEETHAM

AMRITAPURI CAMPUS

KOLLAM 690525

NOVEMBER 2026

AMRITA VISHWA VIDYAPEETHAM
DEEMED TO BE UNIVERSITY UNDER SECTION 3 OF UGC ACT, 1956

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Amrita School of Computing, Amritapuri Campus

BONAFIDE CERTIFICATE

This is to certify that the project report entitled "Secure and Role-Aware Explainable Occupant Tracking in Multi-Building Indoor Smart Spaces", submitted by Ajith Ashok (AM.U4.U4CSE23204), Arjun Rajesh (AM.U4.U4CSE23208), D Aditya Kiran (AM.U4.U4CSE23215) and Yadhu Vipin Madhilakath Madathil (AM.U4.U4CSE23270), in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering at Amrita Vishwa Vidyapeetham, is a bonafide record of the work carried out by them under my guidance and supervision at Amrita School of Computing, Amritapuri, during Semester 7 of the academic year 2026 - 2027.

Dr. Lakshmi Mohan
Project Guide
Signature _____

Project Coordinator
Signature _____

Chairperson, CSE
Signature _____

Reviewers
Signature _____

Place: Amritapuri
Date: 4 November 2026

AMRITA VISHWA VIDYAPEETHAM
DEEMED TO BE UNIVERSITY UNDER SECTION 3 OF UGC ACT, 1956

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Amrita School of Computing, Amritapuri Campus

DECLARATION

We, Ajith Ashok (AM.U4.U4CSE23204), Arjun Rajesh (AM.U4.U4CSE23208), D Aditya Kiran (AM.U4.U4CSE23215) and Yadhu Vipin Madhilakath Madathil (AM.U4.U4CSE23270), hereby declare that the project entitled "Secure and Role-Aware Explainable Occupant Tracking in Multi-Building Indoor Smart Spaces" is a record of our original work carried out under the guidance of Dr. Lakshmi Mohan, Assistant Professor, Department of Computer Science and Engineering, Amrita Vishwa Vidyapeetham, Amritapuri. To the best of our knowledge, this work has not formed the basis for the award of any degree, diploma, fellowship or similar qualification to any candidate at any university. All sources and contributions of others have been appropriately acknowledged.

Place: Amritapuri
Date: 4 November 2026

Student details	**Signature of the student**
Ajith Ashok AM.U4.U4CSE23204	_____
Arjun Rajesh AM.U4.U4CSE23208	_____
D Aditya Kiran AM.U4.U4CSE23215	_____
Yadhu Vipin Madhilakath Madathil AM.U4.U4CSE23270	_____

Project Guide: Dr. Lakshmi Mohan

Signature of the Project Guide: _____

ACKNOWLEDGEMENT

We thank our guide, Dr. Lakshmi Mohan, for the Distributed State Transition System framework on which this work builds and for guiding every review, from the first literature survey to the final evaluation. Her comments at the mid-review shaped the role-based access control, the explainability and the security work reported here.

We are grateful to the Department of Computer Science and Engineering and the Amrita School of Computing, Amritapuri Campus, for the computing facilities and the review process that kept the project on schedule. We also thank the authors of the open datasets and models we used, CASIA-WebFace and the InsightFace models, whose public release made this evaluation possible.

Finally, we thank our team members for the shared effort across the parts of the project: the routing and pipeline, the query, access-control and security layers, the explanation layer and the dashboard.

ABSTRACT

A smart space is an indoor environment equipped with sensors, IoT devices and intelligent infrastructure that collect indoor location data. Multi-building smart spaces, such as university campuses, hospitals and military bases, must identify and follow occupants as they move between buildings, without asking anyone to stop or tap. The usual design keeps every reference template in one central store, which is simple and accurate but concentrates sensitive biometric data. Keeping each building's templates local avoids this, but the capturing building then cannot tell which building holds a visitor's identity, and the base DSTS work resolves visitors through a central master list; among the works reviewed, none treats access control and explanation of answers together with identification. This work does not claim that a decentralised design is better. It asks whether a decentralised alternative can do the same job, and how much recognition accuracy it gives up. Building on the Distributed State Transition System (DSTS) framework, each building keeps its own gallery and publishes only a Bloom filter of locality-sensitive-hash codes of its occupants' face embeddings; a capture that is not recognised locally is routed to a short list of likely buildings, which verify it by angular-distance voting. Because the same tracking data would let any user who can query it follow anyone else, access is governed by two-layer role-based access control: what a requester sees about a person is filtered by the requester's role, from no information, through presence only, up to the exact zone and full trajectory. Every answer comes with an explanation of the rule applied, the evidence behind it, any uncertainty and what was withheld, redacted to the same level so that it never reveals more than the answer itself. A dynamic visitor database, probabilistic zone-level tracking and sealed inter-building transfers complete the system. It is evaluated on 500 simulated occupants (CASIA-WebFace) in 10 buildings, with 10,000 events per simulated day, using a central search as the reference and averaging over 50 random populations. The decentralised pipeline identifies 95.0% of events, against 95.2% for the central search (0.16 percentage points lower), while consulting 11.0% of the galleries; the building-level state F1 is 0.961. Each answer carries a warning when it may be wrong. The warning appears on 74% of the wrong identifications and, as a false alarm, on 2.3% of the correct ones. Building the warnings adds about 13% to the processing time of a query, measured on a single machine, so network delay is not included. These results describe the accuracy that decentralisation retains, not an advantage over a central design. The network, cameras and certificate authority are simulated, so the work demonstrates the design and its accuracy, not a hardened deployment.

Keywords: Occupant identification and tracking; Distributed state transition system; Locality-sensitive hashing; Bloom filter; Role-based access control; Explainable queries

Table of Contents

Acknowledgement i

Abstract ii

List of Figures v

List of Tables vi

1 Introduction 1

1.1 Background and motivation 1

1.2 Objectives 2

1.3 Scope and constraints 2

1.4 Project report organization 4

2 Problem Definition 5

2.1 Problem statement 5

2.2 Existing system and limitations 5

3 Literature Review 6

3.1 Review scope and selection 6

3.2 Descriptive review of literature works 6

3.3 Comparison of reviewed literature 13

4 Requirements 16

4.1 Functional requirements 16

4.2 Non-functional requirements 17

4.3 Hardware requirements 17

4.4 Software and data requirements 19

5 Proposed Methodology **20**

5.1 Design (architecture) or conceptual framework 20

5.2 Proposed approach and procedure 22

6 Results and Analysis **28**

6.1 Evaluation setup 28

6.2 Evaluation criteria 28

6.3 Result 29

6.4 Comparative Analysis 35

6.5 Discussion and Interpretation 37

6.6 Limitations 37

7 Conclusion **39**

7.1 Conclusion 39

7.2 Future work 39

References **41**

A Source Code and Reproducibility **44**

A.1 Setup and execution 44

A.2 Supporting material 45

List of Figures

5.1 System architecture: building nodes, secure communication layer, access control layer and user interface 21

5.2 Inter-building tracking workflow and querying and access workflow 21

6.1 Overview with a simulation running: clock, progress, counts and a live feed with photos; the sign-in (role, building, person) is at the top right 35

6.2 The track of one person across two buildings, with the photo captured at each sighting 36

6.3 The same query from a Dean (left), answered with its "Why this answer" card (rule, evidence, uncertainty, withheld), and from a Student (right), refused with a rule code and no explanation 36

List of Tables

1.1 Objectives, evaluation methods and deliverables 3

3.1 Comparison of reviewed literature 13

4.1 Functional requirements 16
4.2 Non-functional requirements 18
4.3 Software, tools and data 19

5.1 What a building keeps and what it shares 20
5.2 The three layers of the visitor database 24
5.3 Disclosure levels 25
5.4 Access model (class-teacher RBAC) 25
5.5 The fourteen query templates supported by the system 26
5.6 Uncertainty flags 26
5.7 Parameters used 27

6.1 Identification accuracy and gallery touched (average of 50 seeds, 10,000 events per run) 29
6.2 Event-level confusion counts and scores (average of 50 seeds; TN, in the millions, omitted) 30
6.3 Visitor identification and tracking (average of 50 seeds) 30
6.4 Verification policies on routed visitor lookups (average of 50 seeds) 31
6.5 Spread of the results over 50 random populations (10,000 events each) . . . 31
6.6 State-level scores against the state ground truth (average of 50 seeds) 32
6.7 Warning flags on identifications (average of 50 seeds) 33
6.8 Automated tests by area 34
6.9 Comparison with the baseline and the approaches in the literature 36

7.1 Objective-wise status 40

Chapter 1

Introduction

1.1 Background and motivation

A *smart space* is an indoor environment equipped with sensors, IoT devices and intelligent infrastructure that collect indoor location data about its occupants, in order to support occupant-specific services such as access, scheduling and safety [6, 7, 25]. Campuses, hospitals, corporate estates and military bases are typical examples. In each of them the space spans many buildings, so the system must keep identifying and following a person as they move from one building to the next, without asking anyone to stop or tap. Indoor tracking with camera networks is well studied [1, 2, 3, 4, 5], but the identity question changes when the space has several buildings: a person registered in one building is a *visitor* in every other, and the building that senses the visitor does not hold that person's reference data.

This project extends the Distributed State Transition System (DSTS) of Mohan and Menon [6], which models a multi-building smart space as states, events and transition functions. DSTS identifies visitors with a *centralised* database: a master list holds every person's reference embeddings, and every unrecognised capture is searched against it. This is simple and accurate, and we do not argue that it is a poor design. Its cost is that it concentrates biometric data, which, unlike a password, cannot be revoked and reissued if it leaks, and that it makes one server a single point of failure and an attractive target. The recognition part of this project therefore treats the centralised search as the reference and asks how well a *distributed* design holds up against it. In the distributed design each building keeps its own reference data, but a visitor's home building is then unknown, so the capturing building must either contact every other building for every unrecognised face or share reference embeddings between buildings. We measure how much recognition accuracy is given up, and how much of the stored data has to be consulted, when this is avoided.

A second concern is the people who query the tracking data. The same data that helps find a lost visitor would let any user who can query it follow any other person. The information returned about a person must therefore be filtered according to the role of the requester,

from no information at all, through whether the person is present, to their exact zone and full movement history, following the principles of role-based access control [20]. The system must also be able to show why an answer was given. Among the works reviewed in Chapter 3, access control over tracking queries and explanation of the answers are not treated together with distributed identification.

This project therefore combines three things in one system: (i) decentralised identification and tracking, in which a building shares only a Bloom filter of hashed locality-sensitive-hash codes until a likely match is found, then sends the capture embedding over a sealed channel to a short list of buildings for verification, and releases a person's reference embeddings only after a match is confirmed; (ii) privacy-aware querying under role-based access control (RBAC) with a two-layer policy; and (iii) explanations that justify both the access decision and the tracking evidence behind each answer. This report covers the complete system, including state tracking, access control, security, and explanations.

Relevant Sustainable Development Goals. The work maps to three goals of the 2030 Agenda [26]: *SDG 9* (target 9.1, resilient infrastructure), because there is no central biometric store to lose; *SDG 11* (target 11.7, safe and inclusive spaces), because occupants can be located for safety while their data is protected from misuse such as stalking; and *SDG 16* (target 16.6, accountable and transparent institutions), because every access decision is audited and every answer is explained.

1.2 Objectives

The objectives, each linked to how it is evaluated and to its deliverable, are given in Table 1.1.

1.3 Scope and constraints

Intended users. The administrators, staff and ordinary occupants of a multi-building space; each sees tracking data only as their role and relationship allow. The roles used in the implementation are defined in Chapter 5. **Included functions.** Face-embedding-based identification of registered occupants and of visitors across buildings; probabilistic zone-level tracking; fourteen query templates (Q1–Q13 and TRACK, the full-trajectory query); role-based disclosure; explanations; secure inter-building exchange; a web dashboard; and a centralised baseline for comparison.

Exclusions and assumptions. The system works on face embeddings produced by a pre-trained model; it does not train a face model or process live video. Cameras, the network between buildings and the certificate authority are simulated in one process, and the test photos are replayed as camera events. Zones are a fixed nine-zone layout per building. Every visitor is registered in some other building; people registered nowhere (unknown visitors) are future work. **Constraints.** All experiments run on a laptop CPU and are reproducible

Table 1.1: Objectives, evaluation methods and deliverables

#	**Objective**	**Evaluation method**	**Deliverable**
1	Extend the DSTS framework to distributed multi-building identification and tracking	Event identification precision and recall; state-level precision and recall over every state (Sections 6.3.1, 6.3.5)	Pipeline, building nodes, building-level tracking databases (BSTS)
2	Provide an RBAC query interface in which each role sees only the people, and the level of detail, that its relationship allows	Every policy cell tested; access matrix checked against the expected decisions (Section 6.3.8)	Two-layer policy, FastAPI server, dashboard
3	Return explainable responses that justify the access decision and the tracking evidence	Faithfulness and no-leakage tests; flag precision and recall against ground truth; latency overhead (Section 6.3.6)	Explanation model, redaction, "Why this answer" card
4	Design LSH + Bloom-filter routing for decentralised visitor lookup	Routing recall; gallery touched per lookup; accuracy against a centralised search; effective false-positive rate (Sections 6.3.1, 6.3.2, 6.3.7)	Enrolment, routing and verification modules
5	Secure the system: sealed transfers, role-based filtering, audit logging	Tamper, replay, wrong-sender and fail-closed tests; login lockout tests (Section 6.3.8)	Sealed-transfer and access-control modules
6	Evaluate tracking precision and recall, communication cost and explanation quality	TP/FP/FN/TN per cohort and level; 50 seeds; centralised baseline; explanation measurements (Chapter 6)	Evaluation scripts and result tables

from a population seed.

1.4 Project report organization

Chapter 2 states the problem, Chapter 3 reviews fourteen related works, and Chapter 4 lists the requirements. Chapter 5 presents the design and its parameters, Chapter 6 the evaluation, results, comparison and limitations, and Chapter 7 the conclusion and future work. References and an appendix on reproducibility follow.

Chapter 2

Problem Definition

2.1 Problem statement

Problem. Identify and track occupants across a wide-area, multi-building indoor environment while giving secure, role-based and explainable access to where they are and have been, and justify both the access decisions and the tracking evidence behind each answer.

Stakeholders and consequences. The stakeholders are the occupants who are tracked, the users who query the system under different roles, and the auditors. A central biometric store exposes every occupant in a single breach; distribution without routing costs a request to every other building per unrecognised face; uncontrolled access enables stalking; unexplained answers cannot be audited.

2.2 Existing system and limitations

Existing indoor tracking systems use cameras, radio signals or both [1, 2, 9]. Edge-camera frameworks localise and track people inside one site [1, 2, 5], and distributed trackers associate targets across cameras with consensus filtering or shared target tables [3, 4]; none says how a person registered elsewhere is identified or who may query the locations.

The DSTS framework [6] models a multi-building environment as states, events and transition functions and resolves visitors through a centralised master list. Bloom filters have been used to protect biometric templates [10], to count people across locations without revealing their faces [11] and to locate the web proxy that holds a cached object [13], but none of the reviewed work combines approximate, similarity-based routing of a face to a building with role-based access control and explanations.

Gap. The project addresses four gaps: (i) identity discovery across separate buildings without sharing galleries; (ii) evidence that LSH with Bloom filters routes biometric visitors accurately; (iii) role-based control over tracking queries; and (iv) explanations for access decisions and tracking answers.

Chapter 3

Literature Review

3.1 Review scope and selection

Fourteen works (references 1–6, 8 and 10–16) were reviewed in twelve groups (Section 3.2); they are cited by their reference number throughout the report. The search covered indoor occupant tracking with camera and RF sensing, multi-camera association, state-transition models of smart spaces, privacy-preserving biometrics and Bloom-filter indexing, and similarity search with locality-sensitive hashing. Search keywords were: "indoor localization camera network edge", "distributed multi-camera multi-target tracking", "smart space state transition", "Bloom filter biometric" and "locality sensitive hashing".

The publication period is 1970 to 2026: recent works (2020–2026) on tracking and privacy-preserving biometrics were chosen for the problem context, and foundational works on Bloom filters and hashing were included because the routing mechanism is built on them. A work was selected if it is a primary source (a journal article, conference paper or preprint), is directly relevant to tracking, identity discovery, hashing-based routing, or the models used in this project, and gives enough detail to describe its approach and evaluation. Works [1]–[5] are the mid-review set; [6] is the base paper that this project extends. The remaining references (the datasets, face models, access-control model and cryptographic standards used in Chapters 4 and 5) are cited where they are used and are not reviewed here.

3.2 Descriptive review of literature works

Each group below gives one connected paragraph covering the problem or objective, the proposed contribution and approach, the study context, the evaluation criteria, the key findings and the limitations that leave a gap for this project. Table 3.1 compares all of them.

3.2.1 Group identification and localisation with privacy-preserving edge cameras

Hegde et al. (2024) in "Indoor group identification and localization using privacy-preserving edge computing distributed camera network" [1] investigate how to find when and where groups form in an indoor therapeutic space of about 1,700 m^2. The work proposes a pipeline that takes noisy positions and orientations of people, estimated from sparsely placed multi-view edge cameras running a lightweight two-dimensional pose detector, and groups them. The study is situated in a therapeutic built environment with seven participants over 35-minute sessions in groups of two to seven. The authors assess it with precision, recall and F1 for group identification and mean absolute error for group localisation. They report F1 of up to about 90% and a localisation error of 1.25 m, with raw images kept off the network because the edge devices extract the features. The limitations are that the unit of identification is a group and not an individual, and that the study covers one space; it says nothing about identifying a person who is registered elsewhere.

3.2.2 Indoor localisation and multi-person tracking with sparse edge cameras

Kwon et al. (2023) in "A feasibility study on indoor localization and multiperson tracking using sparsely distributed camera network with edge computing" [2] ask whether people can be localised and tracked over a large indoor site with few, low-cost cameras. They deploy 39 edge camera units, each running pose estimation on a Google Coral accelerator, in a therapeutic space of roughly 1,700 m^2, and track people from the pose output. The evaluation reports localisation error, multiple-object tracking accuracy, tracking precision, identity switches and body-orientation error, with an average localisation error of about 1.4 m and a tracking accuracy of about 88.6%. The framework tracks anonymous people inside one site and does not attempt cross-building identity discovery, access control or explanations.

3.2.3 Distributed multi-camera multi-target association

Yang et al. (2022) in "Distributed multi-camera multi-target association for real-time tracking" [3] address real-time association of targets across cameras without a central server. They propose a target-management module in which cameras share a data structure of target labels and appearance information so that detections in different views can be matched, without camera calibration. The evaluation uses several multi-camera benchmarks and an own mobile-camera dataset and reports multi-camera tracking accuracy, IDF1, false positives, false negatives and identity switches. The work improves association, but it keeps no probabilistic state of who is in which building and does not identify where a visitor comes from.

3.2.4 Distributed multi-target tracking in camera networks

Casao et al. (2021) in "Distributed multi-target tracking in camera networks" [4] study tracking of several people with a network of cameras that exchange information only with neighbours. Their approach combines a distributed-consensus Kalman filter with a re-identification network and a distributed tracker manager that handles cross-camera association. They compare against centralised trackers on public multi-camera datasets using tracking accuracy, IDF1-type scores and bandwidth, and report better accuracy and lower bandwidth than the centralised methods they compare with. The gap is that the system tracks within one camera network and has no model of buildings, registered occupants and visitors.

3.2.5 Real-time multi-person multi-camera tracking at the edge

Gaikwad and Karmakar (2021) in "Smart surveillance system for real-time multi-person multi-camera tracking at the edge" [5] build an end-to-end pipeline of person detection, tracking and re-identification that runs in real time on edge devices and follows people across multiple cameras. The pipeline shows that edge tracking and re-identification are practical, but it has no formal multi-building state or query model and no decentralised handling of visitors.

3.2.6 Modelling large-scale camera networks: the DSTS framework

Mohan and Menon (2020) in "Modelling large scale camera networks for identification and tracking: an abstract framework" [6] model identification and tracking in a wide-area camera network as a distributed state transition system. A state records the probability that each occupant is in each zone, an event is a recognition at a zone, and a transition function updates the state when an event occurs; each building has a building-level state transition system (BSTS). Registered occupants are recognised against their own building's database, and visitors are resolved through a centralised master list. The framework is evaluated on a two-building campus with 35 registered occupants, using state-based precision and recall. The visitor step is the part this project replaces: the central list concentrates biometric data and every visitor lookup depends on it. The model of states, events and transitions builds on earlier work on the three Rs of cyber-physical spaces [7].

3.2.7 Face recognition and tracking for security surveillance

Nair et al. (2021) in "Face recognition and tracking for security surveillance" [8] investigate identifying and following faces across surveillance cameras, including cameras with non-overlapping views. The work combines face detection and recognition with tracking so that a person recognised in one camera can be followed into the next. It shows that face recognition is a practical identity cue for tracking, which is the assumption this project relies on. As a surveillance system it is not designed around separate per-building galleries, cross-building identity discovery or role-based querying.

3.2.8 Bloom filters for iris biometrics

Rathgeb et al. (2014) in "On application of Bloom filters to iris biometrics" [10] investigate how Bloom filters can hold iris templates. Representing the template as a Bloom filter makes comparison alignment-free and one-way, so that the stored template protects the biometric while still allowing identification, and the structure supports faster searches than a linear scan of templates. The work establishes that Bloom filters are a usable representation for biometric data. It targets iris codes, not face embeddings, and it protects a single database rather than routing a query among many databases.

3.2.9 Privacy-preserving face-based crowd monitoring

Marzani et al. (2026) in "Head count: privacy-preserving face-based crowd monitoring" [11] address counting individuals across locations without revealing their faces. They combine face features, fuzzy extractors and homomorphically encrypted Bloom filters so that the same person seen in different places is counted once while the underlying biometric stays hidden. This is the closest related work to using Bloom filters for cross-location face matching. Its purpose is counting, not identification and tracking of named occupants, and it has no role-based query layer; this project uses LSH codes and per-building Bloom filters to route a capture towards the building that holds the reference.

3.2.10 Space/time trade-offs in hash coding

Bloom (1970) in "Space/time trade-offs in hash coding with allowable errors" [12] introduces the Bloom filter: a bit array and several hash functions that answer "probably in the set" or "definitely not in the set", with a controllable false-positive rate and no false negatives. The analysis trades a small error probability for a large saving in space. This is the property the project depends on: each building can publish a compact, one-way summary of its occupants. The filter answers exact membership only, which is why locality-sensitive hashing is needed to make nearby embeddings collide; and, as Section 6.3.7 shows, the false-positive estimate assumes queries unrelated to the stored set, which LSH codes of similar faces are not.

3.2.11 Summary Cache: Bloom filters for wide-area cache sharing

Fan et al. (2000) in “Summary Cache: a scalable wide-area web cache sharing protocol” [13] study how web proxies can share cache contents without flooding each other with queries. Each proxy keeps a Bloom-filter summary of the objects held by its peers and contacts only the peers whose summaries match, which sharply reduces inter-proxy messages at a small cost in false hits. The evaluation is on trace-driven simulations of proxy traffic. The same idea of summaries that tell where an item probably lives is applied here to faces, but a face capture never matches exactly, so the summaries must be built from similarity-preserving codes.

3.2.12 Similarity-preserving hashing: SimHash and multi-probe LSH

Charikar (2002) in “Similarity estimation techniques from rounding algorithms” [14] shows that random-hyperplane hashing maps vectors to bits so that the probability of two vectors agreeing on a bit decreases as the angle between them grows, which makes binary codes preserve angular similarity. Lv et al. (2007) in “Multi-probe LSH: efficient indexing for high-dimensional similarity search” [15] improve recall without adding hash tables by also probing nearby buckets chosen from the query, reaching the same recall with far fewer tables; this project applies the idea to binary codes by flipping the least-confident bits. Kirsch and Mitzenmacher (2006) show that Bloom-filter positions can be derived by double hashing with no loss in asymptotic false-positive rate [16]. These three works supply the encoding, multi-probe lookup and hashing used in this project. They are generic indexing techniques and say nothing about biometric privacy or access control.

3.3 Comparison of reviewed literature

Table 3.1 compares the reviewed works on the attributes used above. The gap that follows from the comparison is stated here, and the table gives the evidence. **Gap.** None of the surveyed works routes a face from a building that does not know it to the building that does, using compact similarity-preserving summaries, and none combines this with role-based access to the tracking data and with explanations of the answers. That combination is the contribution of this project.

Table 3.1: Comparison of reviewed literature

Work and citation	**Problem or objective**	**Approach and contribution**	**Study context**	**Evaluation criteria**	**Key findings**	**Limitations and gap**
Work 01 [1] Hegde et al., 2024	Find when and where groups form indoors	Pose estimation and clustering on edge cameras	One therapeutic space, 7 people, 35 min	Precision, recall, F1, MAE	F1 up to about 90%, MAE 1.25 m	Groups, not individuals; one site
Work 02 [2] Kwon et al., 2023	Localise and track people with few cameras	39 edge units with pose estimation	About 1,700 m^2 therapeutic space	Localisation error, MOTA, MOTP, ID switches	About 1.4 m error, MOTA about 88.6%	Anonymous tracking in one site
Work 03 [3] Yang et al., 2022	Distributed multi-target association	Shared label and appearance table, no calibration	Benchmarks and an own mobile-camera set	MOTA, IDF1, FP/FN, ID switches	Real-time association without a server	No building-level state or visitor origin

Work and citation	Problem or objective	Approach and contribution	Study context	Evaluation criteria	Key findings	Limitations and gap
Work 04 [4] Casao et al., 2021	Distributed multi-target tracking	Consensus Kalman filter, re-ID and tracker manager	Public multi-camera datasets	MOTA/IDF1-type, bandwidth	Better than centralised in accuracy and bandwidth	One camera network; no occupants or visitors
Work 05 [5] Gaikwad and Karmakar, 2021	Edge multi-person multi-camera tracking	Detection, tracking and re-ID on edge devices	Multi-camera surveillance video	Tracking accuracy, real-time speed	Real-time edge tracking and re-ID is practical	No multi-building state or query model
Work 06 [6] Mohan and Menon, 2020	Model camera-network identification and tracking	Abstract DSTS: states, events, transitions	Two-building campus, 35 registered occupants	State-based precision and recall	Formal model with building-level BSTS	Visitors via a central master list
Work 07 [8] Nair et al., 2021	Face recognition and tracking	Recognition and tracking across cameras	Surveillance cameras	Recognition and tracking quality	Face is a workable identity cue	Not built around per-building galleries or access control
Work 08 [10] Rathgeb et al., 2014	Protect iris templates	Bloom-filter templates	Iris databases	Comparison and speed	Alignment-free, one-way templates	Iris, not face; one database, no routing
Work 09 [11] Marzani et al., 2026	Privacy-preserving crowd counting	Face features, fuzzy extractor, encrypted Bloom filters	Multi-location counting	Counting accuracy, privacy	Counts people without revealing faces	Counts, does not identify; no roles

Work and citation	Problem or objective	Approach and contribution	Study context	Evaluation criteria	Key findings	Limitations and gap
Work 10 [12] Bloom, 1970	Compact set membership	Bit array and hash functions	Theory	False-positive rate, space	No false negatives; tunable error	Exact membership only; assumes queries unrelated to the stored set
Work 11 [13] Fan et al., 2000	Share web caches cheaply	Bloom-filter summaries of peers	Proxy traces	Messages, hit ratio	Far fewer inter-proxy messages	Exact keys, not similar faces
Work 12 [14, 15, 16] Charikar, 2002; Lv et al., 2007; Kirsch and Mitzenmacher, 2006	Similarity-preserving hashing	Random hyperplanes; multi-probe lookup; double hashing	Theory, similarity search	Recall, tables needed	Angle-preserving codes; fewer tables	Generic; no privacy or access control

Chapter 4

Requirements

4.1 Functional requirements

Table 4.1 lists the functional requirements with their actors, inputs and outputs, and the acceptance criterion used to check each one.

Table 4.1: Functional requirements

ID	**Requirement**	**Actor**	**Input → output**	**Acceptance criterion**
FR1	Recognise a registered occupant against the building's own gallery	Camera event	Embedding → identity or abstain	Accepted when at least 12 of 20 references agree within 80.43°
FR2	Route an unrecognised capture to likely buildings with Bloom filters, then verify there	Building node	Embedding → shortlist, then identity	Shortlist of at most 5 buildings; only they receive the capture
FR3	Update the person's zone probabilities after each identified event	System	Identified event → nine zone probabilities	Nine rows per event summing to 1
FR4	Remember visitors for a day and promote frequent visitors	System	Arrivals → day cache, permanent pool	Repeat sightings need no routing; promotion after 3 entries in 7 days or 8 in 30
FR5	Open and close a visit pointer at the home building	System	Entry and exit events → pointer record	Pointer minted at arrival, closed at departure; no biometrics stored

ID	Requirement	Actor	Input → output	Acceptance criterion
FR6	Answer the fourteen query templates (Q1–Q13 and TRACK, Table 5.5)	Every role	Query → answer	Answer consistent with the stored events
FR7	Apply a two-layer policy before any answer	Query service	Caller, target, purpose → permit or deny	Every denied request returns a reason code and no data
FR8	Return an explanation with each answer, redacted to the caller's level	Query service	Answer → explanation	Explanation never shows more than the answer; faithful to it
FR9	Seal every cross-building message	Building node	Message → sealed envelope	Tampered, replayed or mis-addressed messages are dropped
FR10	Show the campus, zones, queries and security log on a dashboard	All roles	API data → web page	Content filtered by the caller's role
FR11	Compare with a centralised search and report TP, FP, FN, TN	Evaluator	Run files → metrics	Metrics per cohort, state level and seed
FR12	Record every authorisation decision in an audit log	Access layer	Decision → log entry	Entry has layer, code, principal and target
FR13	Sign users in as a demo role or with a per-person credential	Users	Credential → signed token	Token expires in one hour; lockout after repeated failures

4.2 Non-functional requirements

Table 4.2 states the quality requirements with the measure used. The result that satisfies each one is given in Chapter 6.

4.3 Hardware requirements

A laptop-class machine is sufficient: an Intel CPU, 16 GB of memory recommended, and no GPU. All face-model inference is done once, offline, on the CPU; the experiments then work on stored 512-dimensional embeddings. No cameras or dedicated edge devices are required because camera events are replayed from the datasets. A full one-day run of 10,000 events,

Table 4.2: Non-functional requirements

Quality	**Requirement**	**Measure**	**Where reported**
Accuracy	Decentralised identification close to a centralised search	Overall accuracy within 1 point; visitor F1 reported	Section 6.3.1
Communication cost	Touch a small part of the galleries per lookup	Gallery touched per lookup (central = 100%)	Section 6.3.1
Privacy	No gallery, id or embedding in a published filter; no plaintext biometric between buildings	Filter contents inspected; sealed-transfer tests; effective false-positive rate	Sections 6.3.7, 6.3.8
Security	Fail closed; resist tampering, replay, impersonation and brute force	Attack tests; lockout after 5 failures in 60 s	Section 6.3.8
Explainability	Explanations faithful to the answer, free of leakage, and informative about uncertainty	Faithfulness cases; flag precision and recall; latency overhead	Section 6.3.6
Reproducibility	Every result regenerable from a seed	Seeded populations; 50 seeds	Section 6.3.4
Performance	Run on a laptop CPU	10,000 events in about 8 minutes	Appendix A.1
Usability	One web page for all roles	Dashboard with role-filtered views	Section 6.3.9

including enrolment, routing, tracking and evaluation, takes about eight minutes on such a machine.

4.4 Software and data requirements

Table 4.3 lists the software, models and datasets, with their sources.

Table 4.3: Software, tools and data

Item	**Details**
Operating system, language	Windows 11; Python 3.12 in a virtual environment
Numerical and data	NumPy, pandas, matplotlib; ONNX Runtime and OpenCV for the one-off embedding extraction
Face models	InsightFace models: SCRFD detector [18] and ArcFace recogniser [17], 512-dimensional output
Routing and state	Random-hyperplane LSH [14], Bloom filter [12], vote-based verifier, SQLite tracking databases (written from scratch)
Security and API	cryptography library (X25519, HKDF, AES-GCM, Ed25519), HMAC tokens, FastAPI and uvicorn, Prometheus client
Front end	Static HTML and JavaScript dashboard served by the API (no build step)
Testing	pytest with 574 automated tests
Main dataset	CASIA-WebFace [19]: 500 identities, 20 reference and 20 test photos each (20,000 embeddings), in 10 buildings of 50 people
Access conditions	CASIA-WebFace is a public research dataset

Dataset partitioning. For each person, the 20 reference photos enrol them in their building and are never used as events; the 20 test photos are the camera events, so no photo is used for both. Buildings are formed by a seeded random permutation of the 500 people into 10 buildings of 50. The Bloom filters and the mean face used for centring come only from reference photos. No model is trained or tuned on the test photos.

Chapter 5

Proposed Methodology

5.1 Design (architecture) or conceptual framework

The environment is a set of buildings. Each *building node* owns its registered occupants' gallery, a registered database and a visitor database (SQLite) holding the zone probabilities and events of its own occupants and of its visitors, and a visitor lifecycle manager. A *secure communication layer* connects the nodes; an *access control layer* sits between the users and the data; and a *query service* applies the policy and builds the response with its explanation. Figure 5.1 shows the architecture and Figure 5.2 the two main workflows: the inter-building tracking workflow (recognise locally, route, verify, update the state) and the querying workflow (authenticate, authorise, retrieve, apply the response policy, explain).

Table 5.1 lists what each building keeps and what leaves it.

Table 5.1: What a building keeps and what it shares

Item	**Content**	**Leaves the building?**
Gallery	20 reference embeddings per registered occupant	Only the matched occupant's 20 references, sealed, after a confirmed match
Bloom filter	Bit array of LSH items, 58,469 bits (7.3 KB), 7 hash positions, no ids or embeddings	Yes, published to all peers; one-way
Registered and visitor databases	Per-occupant zone probabilities and events, for own occupants and for visitors	No
Visit-pointer database	Visit pointers of own occupants who are away: occupant id, current building, entry and exit times	Read by location queries; contains no biometrics
Visitor entry history and permanent store	Entry history and the embeddings of promoted frequent visitors	No
Day cache	A visitor's 20 references, in memory for one simulated day	No; zeroed at day rollover

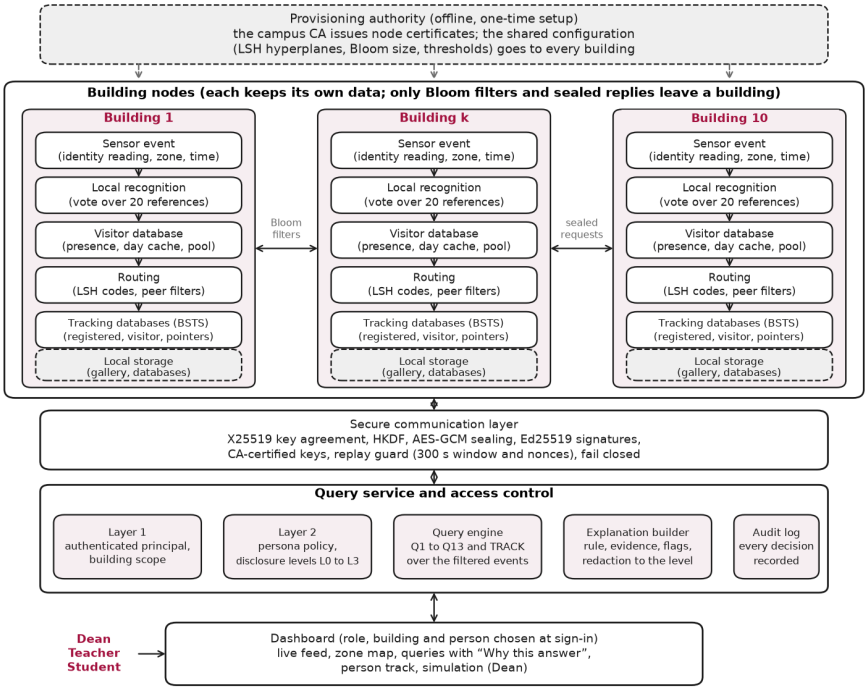


Figure 5.1: System architecture: building nodes, secure communication layer, access control layer and user interface

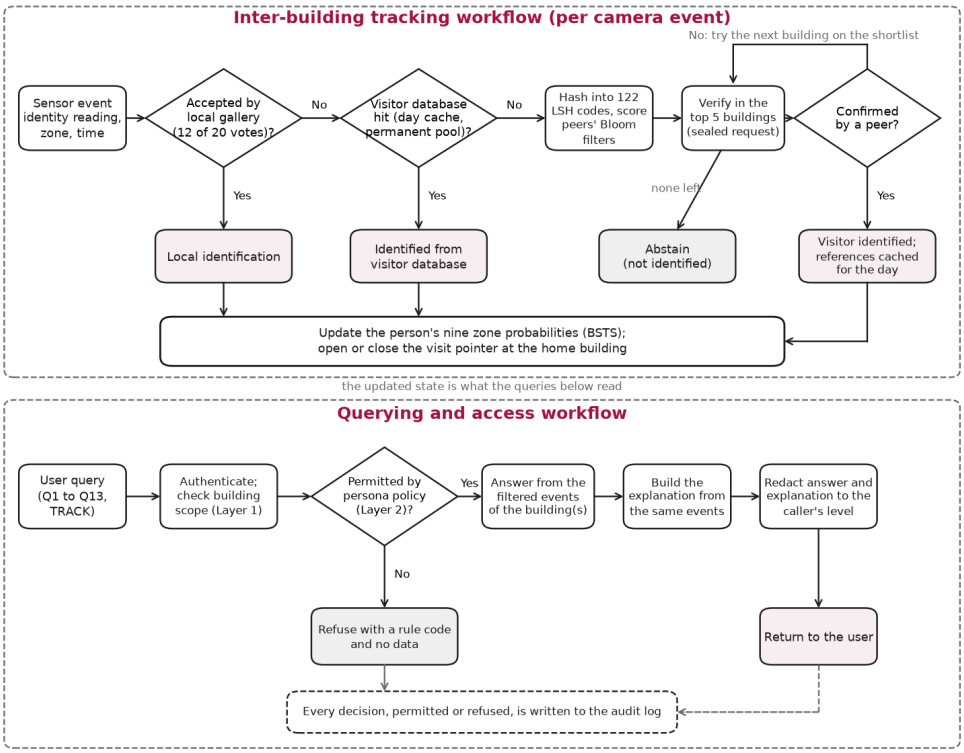


Figure 5.2: Inter-building tracking workflow and querying and access workflow

5.2 Proposed approach and procedure

5.2.1 Embedding preprocessing and enrolment

Faces are detected and aligned with SCRFD [18] and embedded with an ArcFace model [17] (512 dimensions). A raw embedding e is normalised, centred with the mean $\bar{x}$ of the reference embeddings, and normalised again:

$$\hat{e} = \frac{e/\|e\| - \bar{x}}{\| e/\|e\| - \bar{x} \|} \tag{5.1}$$

The $R = 20$ preprocessed references of an occupant u are averaged into a unit-norm centroid:

$$z_u = \frac{\sum_{r=1}^{R} \hat{e}_{u,r}}{\left\| \sum_{r=1}^{R} \hat{e}_{u,r} \right\|} \tag{5.2}$$

The centroid is used only to build the Bloom filter. The 20 individual references are kept in the building for voting, because a single centroid discards the variation between photos of the same person, which the vote relies on.

5.2.2 LSH codes and the Bloom filter

Random hyperplanes (one Gaussian vector w for each slot l and bit k, drawn once and shared by all buildings) define $L = 122$ slots of $K = 11$ bits [14]. The bit h is the side of the hyperplane on which the embedding lies, and the K bits of slot l form an integer code c_l:

$$h_{l,k}(\hat{e}) = \mathbf{1}\left[w_{l,k}^{\mathsf{T}} \hat{e} \geq 0 \right], \qquad c_l = \sum_{k=0}^{K-1} h_{l,k}\, 2^k \tag{5.3}$$

Each occupant's centroid yields L items, the item for slot l being $l \cdot 2^K + c_l$, so a code counts as a match only in the same slot. The items of all occupants of a building are inserted into a Bloom filter [12] with positions derived by double hashing [16]. The filter is sized for 50 occupants at a target false-positive rate of 1%, which gives 58,469 bits and 7 hash positions; with $50 \times 122 = 6{,}100$ items the fill ratio is about 51%. Only the packed bits and a few scalars are published (7.3 KB, or about 10 KB as a file): no ids, embeddings or centroids. Section 6.3.7 measures how close the real false-positive rate is to the 1% target.

5.2.3 Local recognition, routing and verification

A capture is first compared with the capturing building's own gallery. For every candidate occupant the angle between the capture and each of the 20 references is computed, and a reference votes if its angle is at most 80.43°. The winner is the occupant with most votes (ties broken by the smallest single angle); it is accepted only with at least 12 votes. Figure 5.2

gives the flow and the listing below the steps.

If the local check does not accept, the capture is hashed in all L slots. As well as the exact code, the 3 least confident bits of each slot are flipped to give extra probes [15]. Each peer's filter is tested, and a building's score is the number of distinct slots that matched, on a 0 to L scale. The highest-scoring buildings form a shortlist. Its members verify the capture over the sealed channel with the same vote rule against their own occupants, following a verification policy (Section 5.2.4). If no shortlisted building confirms the capture, it is not identified (abstain).

```
identify(capture q, building b):
  r = preprocess(q)
  (who, votes) = vote(own_gallery(b), r)          # >= 12 of 20 within 80.43
  if votes >= 12: return who                      # local occupant
  codes, margins = lsh_encode(r)                  # 122 slots of 11 bits
  score[c] = distinct slots of c's Bloom filter that match any probe
  for c in shortlist(score):                      # sequential policy: best score first
      reply = sealed_request(c, r)                # only c sees the capture
      if reply.matched: return reply.occupant     # references come back sealed
  return None                                     # abstain
```

5.2.4 Shortlist verification policies

The shortlist has a fixed size S (here $S = 5$), and the shortlisted buildings can be asked in more than one way. Three verification policies were considered, which trade the number of buildings contacted against the risk of confirming the wrong person.

- **Verify all.** All S buildings verify the capture and the building whose occupant gathers the most votes wins. It never depends on the ranking, but every lookup contacts S buildings.
- **Sequential.** The buildings are contacted in rank order and the first confirmation ends the search. It contacts the fewest buildings, but a wrongly ranked building that confirms first is accepted without the others being asked.
- **Adaptive.** The gap g between the best and the second-best Bloom score decides: if $g \leq g_0 = 5$ (the top buildings are hard to tell apart) all S buildings verify, otherwise the buildings are asked sequentially. It spends extra contacts only where the ranking is uncertain.

The pipeline can run any of the three. The headline results of Chapter 6 use the *sequential* policy, so their accuracy includes the occasional wrong confirmation by a lower-ranked building, which the "lower-ranked building verified" uncertainty flag marks (Table 5.6). Section 6.3.3 compares the three policies.

5.2.5 Tracking: probabilities and state

For an identified event with candidate distances d_i to each candidate occupant i, the occupant probabilities follow Definition 3.3 of the DSTS framework [6]: with σ the population standard deviation of the distances,

$$p_i = \frac{\exp\left(-d_i^2/2\sigma^2\right)}{\sum_l \exp\left(-d_l^2/2\sigma^2\right)} \tag{5.4}$$

The result is uniform over ties when $\sigma = 0$. The Bloom score, vote counts and ground truth never enter this probability. The building's registered and visitor databases keep, for each occupant, a probability over the nine zones and updates it from each event. The zones are z1 Entrance, z2 Mail room, z3 Office, z4 Lounge, z5 Conference, z6 Classroom, z7 Cafeteria (reached only from the classroom), z8 Exit, and zT, the transition zone to the outdoors; movement between buildings must pass through zT. The event is stored in the building that observed it: registered occupants in the registered database and visitors in the visitor database. The state is reset at the start of every simulated day.

5.2.6 Dynamic visitor database and visit pointers

A visitor is remembered in three layers (Table 5.2). A *transition-zone rule* keeps a visitor who is waiting in zT out of other people's probability updates until they are seen outside zT. A *visit pointer* (occupant id, current building, entry and exit times, no biometrics) is minted at the home building when the visitor arrives and closed at departure, so a location query follows the pointer instead of asking all buildings.

Table 5.2: The three layers of the visitor database

Layer	**Lifetime**	**Content**	**Used for**
Presence	Memory, until the visitor leaves or the day ends	Who is inside and in which zone	Which rows the probability update covers
Day cache	Memory, one simulated day; survives departure; zeroed at rollover	The visitor's 20 reference embeddings	Matching later captures the same day with no routing
Entry history and permanent pool	Disk: entry history and permanent-visitor store	Every arrival; embeddings of frequent visitors	Recognising frequent visitors on any later day without routing

An arrival is the zT event of a visitor who is not inside. When a visitor's *earlier* entries into a building already satisfy a rule from the configuration (default: three entries in seven days, or eight in thirty), the *next* entry stores their embeddings permanently. From then on the visitor is matched from the permanent pool, on every day, without contacting any other building. A wrong identity stored in the cache persists for the rest of the day; Section 6.3.6 reports how rarely such errors are flagged.

5.2.7 Role-based access control

Access control follows the role-based model of Sandhu et al. [20], extended with the relationship between the caller and the target. Every query passes two layers before execution. *Layer 1* authenticates the principal and applies the building scope. *Layer 2* is the persona policy, which returns a disclosure level for the pair (caller, target) and the purpose and time of the query. The levels are shown in Table 5.3 and the access model in Table 5.4. There are 1 Dean, 10 Teachers and 39 Students per building, and each Teacher has a class roster of about four students.

Table 5.3: Disclosure levels

Level	**Name**	**What the caller sees**
L0	None	Nothing; the request is refused with a reason code
L1	Presence	Availability only (defined in the policy; no current rule grants it)
L2	Precise current	Exact zone, zone label, confidence and exact time, for the present only
L3	Historical track	Full trajectory and dwell times

Table 5.4: Access model (class-teacher RBAC)

Role	**May follow**	**Disclosure**	**Cannot see**
Dean (1 per building)	Teachers and students of their own campus (wherever they are), and the Deans of other buildings	L3 for both	Teachers and students of other campuses, including while they visit the Dean's building
Teacher (10 per building)	Only the students on their class roster, in any building	L3 for roster students	Unassigned students, peer teachers, the Dean
Student (39 per building)	Themselves in full; the teachers assigned to them	L3 for self, L2 for assigned teachers	Peer students, other teachers, the Dean

Building-level queries (Q1, Q2) return aggregates to everyone in their own building and the list of people only to a Dean; zone queries (Q10, Q12, Q13) filter each listed occupant through the same policy; students cannot run zone queries. There are fourteen query templates, Q1–Q13 and TRACK (the full trajectory), listed in Table 5.5. Q6 is a point-in-time location query whose answer is redacted to the caller's level; Q3, Q4, Q5, Q7, Q8, Q9, Q11 and TRACK need the full-history level and are refused below it.

5.2.8 Explainable responses

Each query builds an explanation from the same filtered event list that it used to compute the answer, never by recomputing afterwards. The explanation holds the decision rule in

Table 5.5: The fourteen query templates supported by the system

Query	**Question**	**Kind**
Q1	Did any occupant stay in building B after time T?	Building aggregate
Q2	Were there more visitors than registered occupants in building B at time T?	Building aggregate
Q3	Did occupant O leave building B before time T?	History of one person
Q4	Did occupant O enter the building between times T1 and T2?	History of one person
Q5	Did occupant O visit all the zones of building B?	History of one person
Q6	Where was occupant O at time T?	Location at a time
Q7	How many zones did occupant O visit between T1 and T2?	History of one person
Q8	At what time did occupant O first visit zone Z?	History of one person
Q9	How long did occupant O spend in zone Z during the first visit?	History of one person
Q10	Who was in zone Z at time T?	Zone occupancy
Q11	Where was occupant O between T1 and T2?	History of one person
Q12	Who was in zone Z between T1 and T2?	Zone occupancy
Q13	Who ever visited zone Z?	Zone occupancy
TRACK	The full historical trajectory of occupant O	History of one person

one sentence, the evidence events with their roles (decisive, last before t, carry-in state, zone visit), what was looked for and not found, uncertainty flags, what redaction withheld, and the visit pointer if one answered the query. Flags come from the identification provenance stored with every identified event (votes, routing rank, Bloom margin, whether the vote winner was also the nearest candidate) and are listed in Table 5.6. A flag is a plain warning that an answer rests on weaker evidence; it does not say that the answer is wrong. The first four flags concern the identification of a person and the last two concern the answer to a query. The identity probability of Equation 5.4 is not used as a confidence signal, because it is computed from the candidate distances alone and ignores the votes and the routing evidence on which an identification is accepted. Before leaving the service the explanation is redacted to the caller's disclosure level, so it never shows more than the answer does.

Table 5.6: Uncertainty flags

Flag	**What it means**
Winner not nearest	The person accepted is not the one whose single photo is closest to the captured face. Acceptance counts matching photos, so a different person with a closer photo means the evidence contradicts itself: the strongest warning of a wrong identification.
Few votes	A bare pass. A person is accepted when at least 12 of their 20 reference photos match; below 16 matches the identification is marginal.
Lower-ranked building verified	A visitor's possible home buildings are ranked and asked in order, and the first to confirm is accepted. Here the confirming building was not the first choice, so the ranking was off and a look-alike elsewhere may have been accepted.
Narrow routing margin	Which building to ask first was a close call: the two best buildings scored within 5 points of each other (out of 122).
Stale observation	The answer to a time question (for example, where a person was at 4 pm) rests on a sighting more than 60 minutes earlier, so the person may have moved.
Pointer-based value	The answer comes from the visitor's home-building record, which says only which building the person entered and when. No sighting supports it, so there is no exact zone.

5.2.9 Secure inter-building exchange

Every message between buildings is sealed. The sender generates an ephemeral X25519 key [22], derives a symmetric key with HKDF-SHA256 [21], encrypts with AES-GCM [24] using the whole header as associated data, and signs the header, nonce and ciphertext with Ed25519 [23]. Replies are bound to their request. A campus certificate authority issues node certificates that bind both the signing key and the key-agreement key; expiry, revocation, subject and pinning are checked. A replay guard accepts a message only within a 300-second window and once per nonce, and runs after the signature check so the timestamp is authenticated. Any failed step yields no reply and nothing is admitted (fail closed).

5.2.10 Parameters

Table 5.7 lists the parameters used. The Bloom filter was configured for a 1% false-positive rate, but Section 6.3.7 shows that the effective rate that routing sees is about 3.5% because LSH codes of different people collide.

Table 5.7: Parameters used

Parameter	**Value**
LSH bits per slot K / slots L	11 / 122
Bloom target false-positive rate (configured)	0.01
Bloom effective false-positive rate (measured, Section 6.3.7)	about 3.5%
References per occupant R	20
Accept angle / minimum votes	80.43° / 12
Shortlist size / extra probes	5 / 3
Verification policy evaluated	sequential, in rank order
Visitor promotion rule	3 entries in 7 days or 8 entries in 30 days
Uncertainty-flag thresholds	few votes below 16; narrow margin below 5; stale after 60 min

5.2.11 API and dashboard

The FastAPI server exposes login, query, tracking, occupancy and explanation endpoints; every data endpoint runs the same authentication, building scope and two-layer policy before it returns anything, and the dashboard is a static page that calls them. A simulation endpoint, open to Deans only, replays the run's recognised events against a simulated clock; a track endpoint returns one person's day, and a photo endpoint serves the captured or reference photo, both under the same policy.

Chapter 6

Results and Analysis

6.1 Evaluation setup

The evaluation uses 500 occupants in 10 buildings: the 500 CASIA-WebFace people dealt into 10 buildings of 50 by a seeded permutation. One simulated day has 10,000 events, one for each of the 10,000 test photos; 8,000 are local (the person is in their home building) and 2,000 are visitor events. Every headline number in this chapter is the *average over 50 random populations* (seeds 0 to 49); each seed places the same people in different buildings and draws a different event day. Where a single run is needed to show detail (the visitor outcome buckets and the screenshots), seed 42 is used and is named as such.

Safeguards against bias. Reference photos enrol people and are never events; test photos are events and are never enrolled. The mean face and the Bloom filters use reference photos only. The centralised baseline is the same pipeline: the same local check, the same visitor cache and the same vote rule. Only the lookup of a visitor's first sighting differs, a direct search of the other buildings' galleries instead of Bloom routing, so any difference is caused by routing and not by a stronger matcher. Ground truth is read only by the evaluation scripts; recognition, routing and tracking receive observable event fields only.

6.2 Evaluation criteria

All identification metrics use the one-vs-rest convention. An item has one true label and at most one predicted label out of N_L labels. **TP**: the predicted label is the true label. **FP**: a wrong label was predicted. **FN**: the true label was not predicted (a wrong prediction or an abstention). **TN**: every other (item, label) pair, that is items $\times N_L - \text{TP} - \text{FP} - \text{FN}$. A wrong identity is therefore both an FP and an FN, and an abstention only an FN.

$$\text{Precision} = \frac{TP}{TP+FP}, \qquad \text{Recall} = \frac{TP}{TP+FN}, \qquad F_1 = \frac{2PR}{P+R} \tag{6.1}$$

Recall equals the share of events identified correctly, so it is also called identification accuracy. Because TN is in the millions, accuracy computed with TN is close to 1 and is not used to compare systems.

Event level. The label is the occupant id (500 labels), reported for all, local and visitor events. **State level.** State s_k is the campus after event k; the label is (building, zone) with 90 labels, plus building (10 labels) and zone (9 labels) alone, and every occupant the system places counts, including visitors updated by other people's events. **Cost.** Gallery touched per lookup is the number of buildings whose gallery is searched divided by the number of buildings (the centralised server is 100%); routing recall is the share of routed visitors whose home building is in the shortlist.

Explanation criteria. A flag is useful if it is carried by wrong answers and not by right ones. For identifications, *share of wrong flagged* is the fraction of wrong identifications that carry at least one flag, and *share of correct flagged* the false-alarm rate. For answers, an answer is wrong when it differs from the same query run over the identified events with their true occupants; flag *precision* is the share of flagged answers that are wrong and *recall* the share of wrong answers that are flagged. Overhead is the extra latency and response size of building the explanation. **Bloom criterion.** The effective false-positive rate is the share of the enrolled items of the other buildings that a building's filter reports as present.

6.3 Result

6.3.1 Identification: decentralised against centralised

Table 6.1 compares the decentralised pipeline with the centralised search, and Table 6.2 gives the confusion counts and scores, all averaged over the 50 seeds.

Table 6.1: Identification accuracy and gallery touched (average of 50 seeds, 10,000 events per run)

Cohort	Events	Centralised accuracy	Decentralised accuracy	Gap (points)	Gallery touched
All events	10,000	95.17%	95.01%	0.16	11.0%
Local	8,000	95.94%	95.94%	0.00	10.0%
Visitor	2,000	92.11%	91.32%	0.79	15.2%

Over all events the decentralised pipeline identifies 95.01% against 95.17% for the central search, a difference of 0.16 points that is positive in every one of the 50 seeds (0.03 to 0.34 points), while each lookup touches 11.0% of the galleries. Local events are identical in the two systems (95.94% in both): a registered person is recognised by the vote in their own building, with no lookup involved, so the two systems cannot differ there. The visitor events lose 0.8 points (91.32% against 92.11%), which is the cost of routing a capture with Bloom

Table 6.2: Event-level confusion counts and scores (average of 50 seeds; TN, in the millions, omitted)

Cohort	**TP**	**FP**	**FN**	**Precision**	**Recall**	**F1**
All events	9,501	155	499	0.984	0.950	0.967
Local	7,675	60	325	0.992	0.959	0.976
Visitor	1,826	96	174	0.950	0.913	0.931

filters instead of searching the other buildings' galleries directly. The standard deviation over the 50 seeds is only 0.11 points (Section 6.3.4).

6.3.2 Routing and the visitor database

Table 6.3 summarises how visitors are identified and the text below breaks one run (seed 42) down by outcome. In that run, of the 2,000 visitor events, 1,335 (about two thirds) are served from the building's visitor database (the same visitor sighted again the same day), 585 are routed, and the remaining 80 are visitors that an occupant of the capturing building wrongly accepted as a local person (a false accept caused by a look-alike among the building's own occupants). The three groups add up to 2,000. Errors among the routed events are mostly routing misses that end in an abstention, and a smaller number of cases in which the right building is reached but the face still does not pass verification.

Table 6.3: Visitor identification and tracking (average of 50 seeds)

Metric	**Value**
Visitor events per run	2,000
Precision / recall / F1 of visitor identification	0.950 / 0.913 / 0.931
Routing recall (home building in the top five)	90.6%
Buildings contacted per routed lookup	2
Visitor events served from the visitor database	1,345 (67.2%, about two thirds)
Visit-pointer precision	98.3%
Visitor recall in the tracked state: building / zone / building and zone	0.950 / 0.937 / 0.903

The detailed run (seed 42) shows how the headline gap should be read. The centralised baseline identified 95.05% of its 10,000 events and the pipeline 94.88%. The exact difference is 17 events out of 10,000, that is 0.17 points. The 50-seed gap in Table 6.1 is 0.16 points.

6.3.3 Shortlist verification policies compared

The three policies of Section 5.2.4 were run on the same 50 populations, from the routing step onwards, with the adaptive threshold $g_0 = 5$; the sequential row is the run used everywhere

else in this chapter. Table 6.4 counts the routed visitor lookups, that is the visitor events that were not served from the visitor database (about 590 per run), and the share of them identified correctly, identified as the wrong person, or left unidentified.

Table 6.4: Verification policies on routed visitor lookups (average of 50 seeds)

Policy	**Correct**	**Wrong person**	**Abstained**	**Buildings contacted**	**Accuracy, all events**
Sequential	83.89%	2.95%	13.16%	2	95.01%
Adaptive ($g_0 = 5$)	84.49%	2.29%	13.22%	3	95.06%
Verify all	84.42%	2.36%	13.22%	5	95.04%

All three leave about 13% of the routed lookups unidentified, because an abstention means that no shortlisted building confirms the capture, which does not depend on the order of asking. They differ in the wrong-person rate: the sequential policy accepts a lower-ranked building that happens to confirm first, and it misidentifies about 0.6 points more of the routed lookups than the other two (2.95% against 2.29% and 2.36%) and identifies about 0.5 to 0.6 points fewer correctly (better in 50 of the 50 populations for the adaptive policy and in 48 for verify-all). The adaptive policy matches verify-all (the paired difference is 0.07 points, with a 95% interval of ± 0.05) while contacting about 3 buildings per lookup instead of 5 (45% fewer); it costs about one more contact than the sequential policy. Over all 10,000 events the policies differ by at most 0.05 points (95.01% to 95.06%), because routed lookups are about 6% of the events, so the choice of policy is a small effect on the headline accuracy. The sequential policy is kept for the other results because it contacts the fewest buildings and its errors are the ones the uncertainty flags are designed to expose.

6.3.4 Stability across 50 random populations

Each of the 50 seeds places the same people in different buildings and draws a different event day. Table 6.5 gives the spread of the main results over the seeds. The averages in this chapter are the means in the table; the spread is small, so the result does not depend on a favourable assignment of people to buildings. The visitor result varies most because it rests on fewer events.

Table 6.5: Spread of the results over 50 random populations (10,000 events each)

Measure	**Mean**	**SD**	**Min**	**Max**	**95% CI ($\pm$)**
Accuracy, all events	95.01%	0.11	94.67%	95.23%	0.03
Accuracy, local events	95.94%	0.09	95.73%	96.16%	0.03
Accuracy, visitor events	91.32%	0.66	89.70%	92.45%	0.19
Routing recall	90.6%	1.0	88.3%	92.6%	0.3
Gap to the centralised search (points)	0.16	0.06	0.03	0.34	0.02

6.3.5 Tracking state against the state ground truth

Table 6.6 reports the state-level scores. After every one of the 10,000 events the whole campus state, that is who is in which building and zone, is compared with the true state recorded by the simulation, and the scores are averaged over the 10,000 states of a run and then over the 50 seeds.

Table 6.6: State-level scores against the state ground truth (average of 50 seeds)

Level	**Labels**	**Precision**	**Recall**	**F1**	**Visitor recall**
Building	10	0.962	0.960	0.961	0.950
Zone	9	0.942	0.940	0.941	0.937
Building and zone	90	0.926	0.924	0.925	0.903

Building-level state, that is knowing which building each person is in after every event, reaches an F1 of 0.961; the zone within a building and the combination are harder because the nine-zone belief is updated from one event at a time. The standard deviation of the building-level F1 over the 50 seeds is 0.004.

6.3.6 Explanation quality

Every answer comes with a "Why this answer" card and, where the system is unsure, a warning flag (Table 5.6). Three questions were asked: do the explanations match the answers, do the flags appear when an answer is wrong, and what do the explanations cost. The results are averages over 50 seeds on the 10-building system, except the cost, which was timed on seeds 0 to 4.

Do the explanations match the answers? Yes. An explanation is built from the same events that produced the answer, and never recomputed. For every query type, tests with a yes, a no and an empty answer (40 cases) check that each piece of evidence is one of the system's own events and that redoing the answer from the evidence gives the same answer. Further tests check that an explanation never shows more than the answer itself may show to that role, and that explanations are never shared between roles. All of them pass (Section 6.3.8).

Do the flags appear when an identification is wrong? A flag is raised when an explanation is built, from facts saved at the moment the person was identified, such as a low vote count or a close call between two buildings. It never uses the true identity. For each group of identifications, Table 6.7 shows how many wrong identifications carry a flag (more is better) and how many correct ones carry one by mistake (fewer is better).

- **Overall,** about 3 in 4 wrong identifications carry a flag and only 2 in 100 correct ones do. A flag therefore means "check this one": 34% of flagged identifications are wrong, against about 0.4% of unflagged ones.

Table 6.7: Warning flags on identifications (average of 50 seeds)

Identified	**Wrong ones with a flag**	**Correct ones with a flag**
All identifications	73.9%	2.3%
In the person's own building	74.2%	0.9%
Visitors found by routing	85.3%	30.0%
Visitors from the visitor database	4.3%	0.0%

- **The weak spot is the visitor database.** A visitor recognised from the day's cache leaves almost no trail for the flags to use, so only 4% of its few wrong identifications (about 11 per day) are flagged, and such an error stays until the end of the day. Routed visitors are flagged reliably but also noisily (30% of their correct identifications carry a flag).
- **The strongest flag is "winner not nearest"** (Table 5.6): 714 of the 730 identifications that carry it, over the 50 seeds, were wrong.

Do the flags carry over to answers? The same check was made on query answers (where a person was at a given time, how many zones they visited, and their full track), on about 6,900 answers per run. 21% of them carry a flag. Of the wrong answers 79% are flagged, and 39% of the flagged answers are wrong, so a flag again means that an answer deserves a second look rather than that it is wrong.

What do explanations cost? Building the explanation makes a query about 13% slower (5.9 ms to 6.6 ms, summed over the fourteen query types and timed on seeds 0 to 4, one at a time) and the response about seven times larger (16 KB to 119 KB with the full explanation), because the evidence is listed.

6.3.7 The Bloom filters' real false-positive rate

Each filter is sized for a 1% false-positive rate on the assumption that its items behave like independent random keys. LSH codes of different people are not independent: two occupants whose embeddings fall on the same side of the same hyperplanes receive the same code in that slot. The effective rate was therefore measured directly, by testing the enrolled items of the nine other buildings against each building's filter and averaging over the buildings and the 50 seeds.

The effective rate is about 3.5 times the configured one. The Bloom array itself is not at fault: random integers that were never inserted are reported at 0.95%, essentially the 1% target. The excess comes from the codes: 2.64% of the items of other buildings already equal an enrolled item in the same slot, close to the 2.44% expected if the codes of 50 occupants were spread evenly over 2048 values. The 50-seed value, measured on 10 buildings, is 3.56%.

Privacy consequence. The filter is one-way, but membership in it can be tested, and a

positive does not prove that a particular face is enrolled in the building: with an effective rate of about 3.5% on top of genuine matches, a positive is ambiguous. That ambiguity limits what a single positive reveals, and routing therefore only ranks buildings and never identifies anyone; identification needs the exact vote on the real references in the shortlisted buildings. The same finding also means that the configured 1% must not be quoted as the system's false-positive rate, and that the privacy argument should rest on the one-way encoding and on the verification step, not on a small false-positive rate. No formal privacy guarantee is claimed.

6.3.8 Verification of the security and policy layers

The security, policy and explanation layers are verified by 574 automated tests (Table 6.8). They include attack cases written for this project: tampering with every header field of a sealed message, replay, wrong sender or recipient, a revoked certificate, brute-force login, a token used as another person, and cache sharing across roles.

Table 6.8: Automated tests by area

Area	**Tests**	**What is checked**
access_control	112	Every policy cell, structured Layer-1 decisions, building scope, lockout, credentials, tickets, API-level RBAC attacks
secure_transfer	61	Sealed payloads: tampering, replay, wrong sender or recipient, revoked certificate, fail closed, strict decoding
api	98	Login and demo sign-in, queries per role, caching, zone and time-frame views, simulation control, person track and photos, dashboard adapter
queries	181	Query engine, multi-day normalisation, explanation faithfulness, leakage and uncertainty, CLI access rules
pipeline, population, events, evaluation	98	Local recognition, routing and tracking stages, visitor database, seeded populations, event generation, ground truth, confusion metrics
standalone, centralised, state library	19	Standalone demo, centralised baseline, state library
shared utilities	5	Dated timestamps, time-window arithmetic
Total	**574**	All passing on the final commit

A persona matrix tool replays 64 requests (Dean, Teacher, Student and an outsider against each query type and target type) and checks the decision, the rule code and the disclosure level of each. A request for a person in another building is denied unless a relationship allows it: a Teacher may follow a rostered student wherever the student is, and a Dean may follow the Deans of other buildings; every other cross-building request is refused, with a rule code that records the reason.

6.3.9 The dashboard

The dashboard demonstrates the access model as much as the tracking. A Dean starts a simulation of one day, one week or one month: the recognised events of the run are replayed against a simulated clock, and the campus map, the occupancy, the live feed and every query follow it (Figure 6.1). The person at the keyboard is chosen by role, building and name (any Dean, Teacher or Student of any building), and what the page shows is then decided only by the two policy layers: the same feed shows a Dean of building 8 every event of that campus, a Teacher only the assigned students, and a Student only themselves.

Choosing a building shows its nine zones; choosing a zone lists who was there in the time frame, named only where the role may see the exact location. Choosing a person opens their day as a track across buildings and zones (Figure 6.2): each photo is the one the camera captured at that moment, a blue dashed line joins sightings inside a building and a red one a move to another building. The track is shown only where the role may see the full history (disclosure L3): a Dean for the people of their own campus and for the other Deans, a Teacher for the students on their roster, and a Student for themselves. A Student sees the exact current position of an assigned Teacher but no track: a location query from a Student is always answered for the present, whatever time is asked, so repeated queries cannot rebuild a history. A Student gets nothing at all about a Dean, a peer or an unassigned Teacher. Photos follow the exact-location level (L2 or higher). A query returns its answer with the "Why this answer" card, and the same query from a Student, who may not follow that person, is refused with a rule code and no explanation (Figure 6.3).

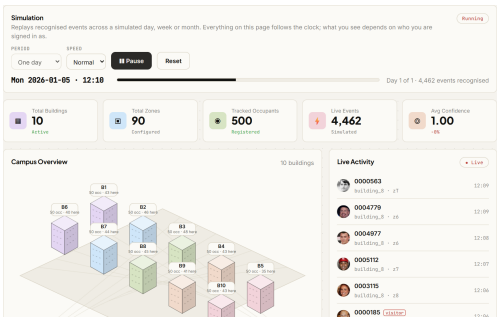


Figure 6.1: Overview with a simulation running: clock, progress, counts and a live feed with photos; the sign-in (role, building, person) is at the top right

6.4 Comparative Analysis

Table 6.9 compares the approaches. The fair comparison for this project is the same pipeline with a central search; other published systems [1, 2, 3, 4, 5] use different data and tasks and cannot be compared number for number.

Routing costs 0.16 points overall and 0.8 points on visitors, in exchange for touching about

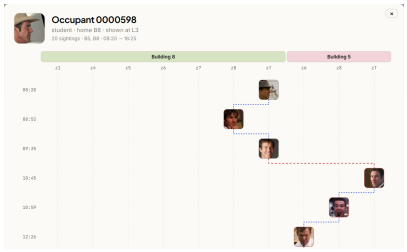


Figure 6.2: The track of one person across two buildings, with the photo captured at each sighting

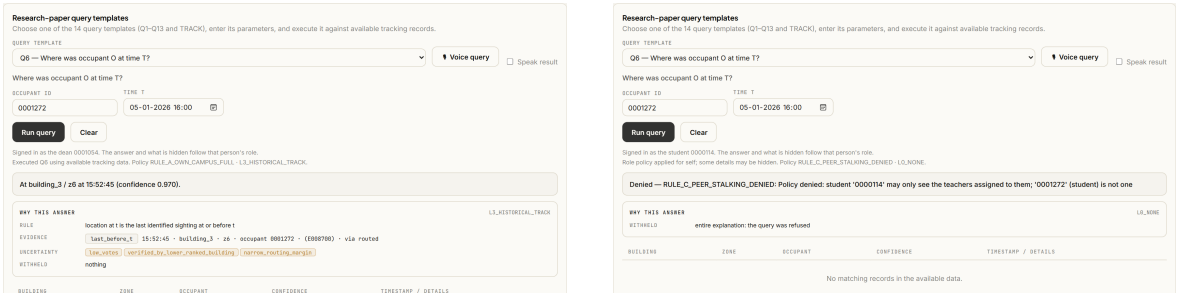


Figure 6.3: The same query from a Dean (left), answered with its “Why this answer” card (rule, evidence, uncertainty, withheld), and from a Student (right), refused with a rule code and no explanation

Table 6.9: Comparison with the baseline and the approaches in the literature

Approach	**Visitor lookup**	**Face data leaving a building**	**Result here**
Centralised search over all galleries (baseline)	Direct search of the other buildings’ galleries	Galleries pooled in one store	95.17%; 100% of the gallery searched
Decentralised LSH + Bloom routing (this work)	Rank by Bloom score, verify the shortlist	Only a 7.3 KB Bloom filter; references move only after a confirmed match	95.01%; 11.0% of the gallery
DSTS base paper [6]	Central master list	Master list of visitor embeddings	Not re-run; replaced by the row above
Edge-camera tracking [1, 2, 3, 4, 5]	Within one site	Not applicable	No cross-building identity, RBAC or explanations

a tenth of the gallery and never pooling it.

6.5 Discussion and Interpretation

The results meet the objectives that can be measured. Decentralised identification is within 0.2 points of a central search overall while touching about 11% of the galleries, so the privacy and locality gain costs little accuracy (objectives 1 and 4). Most visitor events need no routing because repeat sightings the same day are served from the day cache, so most visitor events contact no other building. Tracking reaches an F1 of 0.961 at building level and 0.941 at zone level (objective 1). The order in which shortlisted buildings are asked matters little for accuracy: verifying all five buildings, or only the uncertain cases, reduces wrong identifications of routed visitors by about 0.6 points compared with asking in rank order, at 3 to 5 contacts instead of 2.

The access model and the explanations are verified by tests, flag scores and a persona matrix (objectives 2, 3 and 5). The flags catch about three quarters of the wrong identifications for about 13 per cent of extra latency, but the visitor database, whose errors are rarely flagged, is the main weakness.

One finding qualifies the design: the effective false-positive rate of the Bloom filters is about 3.5%, not the configured 1%, so the privacy argument must not rely on a small false-positive rate.

6.6 Limitations

- The network, cameras and the face detector are simulated. Arrays are plaintext in memory before sealing and after opening, and the certificate authority is a simulated substitute whose signature is a message authentication code, not a real public-key infrastructure.
- The API has no TLS unless a reverse proxy adds it, the stored embeddings and tracking databases are not encrypted, and token secrets, caches and lockout counters are per process. The dashboard's demo mode lets any viewer act as any role, by design. No outside penetration test was done; the attack tests are those written by the authors.
- Recognition of people wearing face masks is not implemented: the system assumes unmasked faces, and masked or otherwise occluded faces were not evaluated.
- A wrong identity stored in the visitor cache persists for the rest of the day and is rarely flagged; a visitor whose departure is not identified stays "present" until the day ends.
- The ArcFace model was trained on large web-collected face data that may overlap the identities of CASIA-WebFace, so recognition of people unseen by the model could be

harder.

- No other routing mechanism, such as a distributed hash table or an approximate-nearest-neighbour index shared between buildings, was evaluated; such alternatives were excluded on privacy and cost grounds, not by measurement.
- No formal privacy guarantee is claimed, and the effective Bloom false-positive rate (about 3.5%) exceeds the configured rate.
- The one-week and one-month simulations in the dashboard replay the recognised events of a single day with shifted times and weekend attendance; they demonstrate the interface and the access rules and are not a longitudinal evaluation.
- Every visitor is registered in some building, so unregistered people cannot be identified. Only 500 people in 10 buildings were measured, and the results come from simulated movement on replayed photographs, not from live video.

Chapter 7

Conclusion

7.1 Conclusion

This project built and evaluated a system that identifies and tracks occupants across buildings without pooling face data, answers queries according to the caller's role and relationship, and explains each answer. A visitor is found by routing a capture with LSH codes and per-building Bloom filters and verifying it only in the shortlisted buildings. On 10,000 events per day, averaged over 50 random populations, it identifies 95.0% correctly against 95.2% for a centralised search, touching 11.0% of the galleries; the 95% confidence interval of the accuracy is ± 0.03 points, and building-level state F1 is 0.961. A dynamic visitor database makes about two thirds of the visitor events free of routing. A two-layer class-teacher policy, sealed transfers and redacted explanations are implemented and covered by 574 automated tests. The uncertainty flags attached to answers catch about 74% of the wrong identifications and 79% of the wrong answers for about 13% of extra latency, although they rarely flag errors that come from the visitor database. Measuring the Bloom filters showed that their effective false-positive rate is about 3.5%, not the 1% configured, so the privacy argument rests on the one-way encoding and on exact verification. All six objectives are achieved in simulation (Table 7.1); the security work is a tested design on a simulated network and not a hardened deployment.

7.2 Future work

Three directions follow directly from the results.

Masked face recognition. The system assumes unmasked faces. Recognising people who wear masks needs a face model trained with masks instead of one trained on unmasked faces, a real mask detector, a few real masked photos per person, and tracking context, such as who is already inside the building. Each can be validated on the same seeds once a masked group of people is added to the simulated population.

Table 7.1: Objective-wise status

Objective	**Planned**	**Status**	**Evidence**
1. Multi-building DSTS extension	Mid review	Achieved	BSTS over 10 buildings; building-level state F1 0.961
2. RBAC query interface	24/9/26	Achieved	Two layers, class-teacher model, every policy cell tested, 64-request persona matrix
3. Explainable responses	26/9/26	Achieved	Faithfulness cases, flag recall 74% on identifications, 13% overhead
4. LSH + Bloom routing	Mid review	Achieved	11.0% of gallery touched, 0.2-point accuracy cost; effective false-positive rate 3.5%
5. Security layer	2/10/26	Achieved (simulated network)	Sealed transfers, tokens, lockout, audit log
6. Evaluation	Final review	Achieved	Event and state TP/FP/FN/TN, 50 seeds, centralised baseline, explanation and Bloom measurements

Unknown visitor re-identification. Today a visitor must be registered in some building, and a face that is registered in no building is left unidentified. A future version should re-identify unknown visitors across buildings and days: keep one-way codes of the unknown faces (LSH with Bloom filters) under a temporary identifier, link their sightings into a track, and let an administrator later name or purge them. This needs retention and privacy rules, and a check that two different strangers are not merged, which can be measured with the same precision and recall used here once unregistered people are added to the simulated population.

Other types of sensor recognition. The pipeline needs only an identity estimate together with a zone and a time. Badge or RFID readers, beacons or other sensors can therefore replace the camera or complement it. Each new sensor needs its own error measurement, and the access rules and the explanations must be checked for it.

References

[1] C. Hegde *et al.*, "Indoor group identification and localization using privacy-preserving edge computing distributed camera network," *IEEE J. Indoor Seamless Position. Navig.*, vol. 2, pp. 51–60, 2024, doi: 10.1109/JISPIN.2024.3354248.

[2] H. Kwon *et al.*, "A feasibility study on indoor localization and multiperson tracking using sparsely distributed camera network with edge computing," *IEEE J. Indoor Seamless Position. Navig.*, 2023, doi: 10.1109/JISPIN.2023.3337189.

[3] S. Yang, F. Ding, P. Li, and S. Hu, "Distributed multi-camera multi-target association for real-time tracking," *Sci. Rep.*, vol. 12, Art. no. 11052, 2022, doi: 10.1038/s41598-022-15000-4.

[4] S. Casao, A. Naya, A. C. Murillo, and E. Montijano, "Distributed multi-target tracking in camera networks," arXiv:2010.13701, 2021.

[5] B. Gaikwad and A. Karmakar, "Smart surveillance system for real-time multi-person multi-camera tracking at the edge," *J. Real-Time Image Process.*, 2021, doi: 10.1007/s11554-020-01066-8.

[6] L. Mohan and V. Menon, "Modelling large scale camera networks for identification and tracking: an abstract framework," *IET Comput. Vis.*, vol. 14, no. 7, pp. 426–433, 2020.

[7] V. Menon, B. Jayaraman, and V. Govindaraju, "The three Rs of cyberphysical spaces," *Computer*, vol. 44, no. 9, pp. 73–79, 2011.

[8] S. P. Nair, K. A. Reddy, P. K. Alluri, and S. Lalitha, "Face recognition and tracking for security surveillance," *J. Intell. Fuzzy Syst.*, vol. 41, no. 5, pp. 5337–5345, 2021, doi: 10.3233/JIFS-189856.

[9] Z. Wu *et al.*, "Learning-based tracking-before-detect for unconstrained indoor human tracking using RF signal," *IEEE Trans. Mobile Comput.*, vol. 24, no. 6, pp. 4777–4793, 2025, doi: 10.1109/TMC.2025.3529501.

[10] C. Rathgeb, F. Breitinger, C. Busch, and H. Baier, "On application of Bloom filters to iris biometrics," *IET Biometrics*, vol. 3, no. 4, pp. 207–218, 2014.

[11] F. Marzani, T. van Ede, G. Heijenk, and M. van Steen, "Head count: privacy-preserving

face-based crowd monitoring," arXiv:2604.14250, 2026.

[12] B. H. Bloom, "Space/time trade-offs in hash coding with allowable errors," *Commun. ACM*, vol. 13, no. 7, pp. 422–426, 1970.

[13] L. Fan, P. Cao, J. Almeida, and A. Z. Broder, "Summary cache: a scalable wide-area web cache sharing protocol," *IEEE/ACM Trans. Netw.*, vol. 8, no. 3, pp. 281–293, 2000.

[14] M. S. Charikar, "Similarity estimation techniques from rounding algorithms," in *Proc. 34th Annu. ACM Symp. Theory Comput. (STOC)*, Montréal, Canada, 2002, pp. 380–388.

[15] Q. Lv, W. Josephson, Z. Wang, M. Charikar, and K. Li, "Multi-probe LSH: efficient indexing for high-dimensional similarity search," in *Proc. 33rd Int. Conf. Very Large Data Bases (VLDB)*, Vienna, Austria, 2007, pp. 950–961.

[16] A. Kirsch and M. Mitzenmacher, "Less hashing, same performance: building a better Bloom filter," in *Algorithms – ESA 2006*, ser. LNCS, vol. 4168, Zürich, Switzerland, 2006, pp. 456–467.

[17] J. Deng, J. Guo, N. Xue, and S. Zafeiriou, "ArcFace: additive angular margin loss for deep face recognition," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Long Beach, CA, USA, 2019, pp. 4690–4699.

[18] J. Guo, J. Deng, A. Lattas, and S. Zafeiriou, "Sample and computation redistribution for efficient face detection," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, virtual event, 2022.

[19] D. Yi, Z. Lei, S. Liao, and S. Z. Li, "Learning face representation from scratch," arXiv:1411.7923, 2014.

[20] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, "Role-based access control models," *IEEE Computer*, vol. 29, no. 2, pp. 38–47, 1996.

[21] H. Krawczyk and P. Eronen, "HMAC-based extract-and-expand key derivation function (HKDF)," RFC 5869, IETF, 2010.

[22] A. Langley, M. Hamburg, and S. Turner, "Elliptic curves for security," RFC 7748, IETF, 2016.

[23] S. Josefsson and I. Liusvaara, "Edwards-curve digital signature algorithm (EdDSA)," RFC 8032, IETF, 2017.

[24] M. Dworkin, "Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC," NIST Special Publication 800-38D, 2007.

[25] M. S. S. Suresh and V. Menon, "A generic and scalable approach to maximize coverage in diverse indoor and outdoor multicamera surveillance scenarios," *IEEE Trans. Syst.,*

Man, Cybern., Syst., vol. 53, no. 2, pp. 1172–1182, 2022.

[26] United Nations General Assembly, "Transforming our world: the 2030 Agenda for Sustainable Development," A/RES/70/1, 2015.

Appendix A

Source Code and Reproducibility

The complete source code, configuration and test suite of the project are in one Git repository: `github.com/yadhu-vipin/Occupant_Tracking_v2`, branch `test_7`. No passwords, tokens or personal data are stored in the repository; the datasets and model files are public and are downloaded separately (see below), and the login master secret is supplied through an environment variable.

A.1 Setup and execution

Dependencies. Python 3.12; the packages in `requirements.txt` (NumPy, pandas, cryptography, FastAPI, uvicorn, pydantic, prometheus_client, PyYAML, httpx, pytest). The face models were used once to extract the embeddings and are not needed to run the pipeline on the stored embeddings.

Steps.

1. Clone the repository, create a virtual environment and run `pip install -r requirements.txt`.
2. Place the embedding corpus files `emb_arcface.npy` (20,000 $\times$ 512) and `meta.csv` in `corpus/`. They are not in Git because of their size.
3. Run the experiment with the commands below; every seed writes into its own folder `runs/seed_XX/`.

```
python manage.py test                          # the automated tests
python manage.py run --seeds 0-49 --days 1     # 50 populations, summary
python -m centralized.compare_all              # central baseline
python -m evaluation.evaluate_explanations     # flags and overhead
python tools/measure_bloom_fpr.py              # Bloom false-positive rate
DSTS_DEMO_MODE=1 python manage.py serve --seed 42   # API, port 8000
```

Configuration. All settings of an experiment are in `config/experiment.yaml`: the population (10 buildings of 50, default seed 42), the event model (by default 14 simulated days with 8 events per occupant per day and a visitor share of 20%; the results in this report use the one-day generator, `-days 1`, which replays every test photograph once and has the same visitor share) and the visitor promotion rule. The federation contract (hash functions, Bloom size, thresholds) is in `shared/params.json` and `shared/routing_params.json`; every artifact carries a hash of it, and a mismatch is rejected.

Time and outputs. A one-day run of 10,000 events takes about eight minutes on a laptop CPU. Each run writes its own folder under runs/seed_XX/ (population, node data, events, results).

A.2 Supporting material

A.2.1 Repository layout and sources of the numbers

The repository is grouped by role: `population`, `events`, `buildinglib` and `recognition` (the building node and the pipeline), `access_control` and `secure_transfer` (the two security layers), `evaluation`, `centralized` and `tools` (metrics, baseline and measurements), `dsts/api` (the server and dashboard) and `tests`. Every number in this report is read from the files a run writes under `runs/seed_XX/results/`, and the 50-seed averages from `runs/summary.json`; the commands in Section A.1 regenerate all of them.

A.2.2 Dataset documentation

CASIA-WebFace subset. 500 identities with 40 embeddings each, split into 20 reference and 20 test embeddings; embeddings were extracted once with the InsightFace buffalo_l models and are stored as 512-dimensional unit vectors.